ROLL.NO:240701506 **NAME: SHREYA KS**

EXPERIMENT - 11
CONTINGOUS MEMORY ALLOCATION

1. First Fit Contiguous Memory Allocation

AIM:

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally display the allocation details and calculate internal fragmentation

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

For each process, the program prints whether the process is allocated or not ○ ○ If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size). If not allocated, it indicates that the process cannot be allocated

Sample Input 1

5
100 500 200 300 600
4
212 417 112 426

Sample Output 1

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88

Process 4 of size 426 is not allocated

CODE:

```c
#include <stdio.h>

int main() {
    int m, n;

    // Input number of memory blocks
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int blockSize[m];           // remaining size after allocations
    int originalBlockSize[m];   // for printing original size

    // Input block sizes
    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        originalBlockSize[i] = blockSize[i];
    }

    // Input number of processes
    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int processSize[n];
```

```c
    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    // First Fit allocation
    for (int i = 0; i < n; i++) {
        int allocated = 0;

        // Search blocks from start (First Fit)
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                int fragment = blockSize[j] - processSize[i];
                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n"
                       i + 1, processSize[i], j + 1,
                       originalBlockSize[j], fragment);
                blockSize[j] -= processSize[i];
                allocated = 1;
                break;  // use first suitable block only
            }
        }

        if (!allocated) {
            printf("Process %d of size %d is not allocated\n",
```

```c
                printf("Process %d of size %d is not allocated\n",
                       i + 1, processSize[i]);
            }
        }

        return 0;
}
```

OUTPUT:

```
5
100 500 200 300 600
4
212 417 112 426
```

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 50
with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 60
with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 50
with Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

2.Best Fit Contiguous Memory Allocation

AIM:

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available error, blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

• For each process, display whether it is allocated or not

○ If allocated, display the block number, block size, and internal fragmentation.
• If not allocated, display that the process could not be allocated.

Example 1
Sample Input 1
5
100 500 200 300 600
4
212 417 112 426

Sample Output 1

Enter number of memory blocks
Enter size of each block:
Enter number of processes:
Enter size of each process
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated |to Block 5 of size 600 with Fragment 174

Example 2
Sample input 2
3
100 200 300
4
90 150,250 400

Sample Output 2

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 90 is allocated to Block 1 of size 100 with Fragment 10
Process 2 of size 150 is allocated to Block 2 of size 200 with Fragment 50
Process 3 of size 250 is allocated to Block 3 of size 300 with Fragment 50
Process 4 of size 400 is not allocated

CODE:

```c
#include <stdio.h>
#include <limits.h>

int main() {
    int m, n;

    // Input memory blocks
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int originalSize[m];
    int availableSize[m];

    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &originalSize[i]);
        availableSize[i] = originalSize[i];  // Initially, available = original
    }

    // Input processes
    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int processSize[n];
```

```
26      printf("Enter size of each process:\n");
27      for (int i = 0; i < n; i++) {
28          scanf("%d", &processSize[i]);
29      }
30
31      // Best Fit allocation with dynamic partitioning
32      for (int i = 0; i < n; i++) {
33          int allocated = 0;
34          int bestFitIndex = -1;
35          int minFragment = INT_MAX;
36
37          // Find the best fit block (smallest block that fits)
38          for (int j = 0; j < m; j++) {
39              if (availableSize[j] >= processSize[i]) {
40                  int fragment = availableSize[j] - processSize[i];
41                  if (fragment < minFragment) {
42                      minFragment = fragment;
43                      bestFitIndex = j;
44                  }
45              }
46          }
47
48          // Allocate to the best fit block if found
49          if (bestFitIndex != -1) {
```

```
47
48          // Allocate to the best fit block if found
49          if (bestFitIndex != -1) {
50              allocated = 1;
51              printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
52                     i + 1, processSize[i], bestFitIndex + 1, originalSize[bestFitIndex], minFragment);
53
54              // Update available size to the remaining fragment
55              availableSize[bestFitIndex] = minFragment;
56          }
57
58          if (!allocated) {
59              printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
60          }
61      }
62
63      return 0;
64  }
```

OUTPUT:

```
5
100 500 200 300 600
4
212 417 112 426
```

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300
with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500
with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200
with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600
with Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```

RESULT:

Thus, the Contiguous Memory Allocation technique was implemented successfully and the memory blocks were allocated to processes correctly.